



**Lab Title:** Using Components with Known Vulnerabilities

**Student Name:** Nebeuwa Ifeanyichukwu Raphael

**Student ID:** 202520850LE

**Course Name:** BVWS103 – Owasp Top 10 Vulnerabilities  
Exploitation Techniques

**Instructor Name:** AMMAR AMINU

**Date of Submission:** 08/02/2026

## **Executive Summary**

In this lab, I learned about the security risks of using software components that have known vulnerabilities. I created a simple web application that used an old version of jQuery, which is not secure. By testing the application, I was able to see how an attacker could inject malicious code and make it run in the browser. After that, I fixed the problem by updating jQuery to a newer version and changing the code to handle user input safely. This lab helped me understand why keeping software components updated is very important for security.

## **Lab Objective**

The objective of this lab was to help me understand how outdated components can create security problems in a web application. I aimed to identify a vulnerable component, exploit the weakness it introduced, and then fix it properly. Through this lab, I learned how simple updates and safe coding practices can prevent common attacks like cross-site scripting (XSS).

## **Lab Environment**

I carried out this lab on my local machine using a web browser and a basic local web server. I created a simple PHP web page and included an old version of jQuery to simulate a vulnerable component. All testing was done locally in a controlled environment, which made it safe to experiment and learn without affecting any real systems or data.

## Task 1: Exploit a Vulnerable Component

### Step 1: Setting up the Vulnerable PHP Application

- Created a new folder ([Lab10\\_VulnerableComponents](#))

```
raph@kali: /opt/lampp/htdocs/lab10_VulnerableComponents
Session Actions Edit View Help
(raph@kali)-[/opt/lampp/htdocs]
$ sudo mkdir lab10_VulnerableComponents
[sudo] password for raph:
Sorry, try again.
[sudo] password for raph:
Sorry, try again.
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs]
$ ls
applications.html      Lab2_HTMLInjection
bitnami.css            Lab4_CommandInjection
dashboard              lab6_SensitiveDataExposure
favicon.ico            lab8_BrokenAccessControl
img                    lab9_SecurityMisconfiguration
index.php              test.php
lab10_VulnerableComponents
lab10_xss              vulnerable_app
lab17_Insecure_Deserialization
week2_php_lab

(raph@kali)-[/opt/lampp/htdocs]
$ cd lab10_VulnerableComponents
```

- Inside the folder, I created the following files
  - Index.php (a page that stimulate the vulnerable web application)

```
(raph@kali)-[/opt/lampp/htdocs/lab10_VulnerableComponents]
$ sudo nano index.php

(raph@kali)-[/opt/lampp/htdocs/lab10_VulnerableComponents]
$ ls
index.php

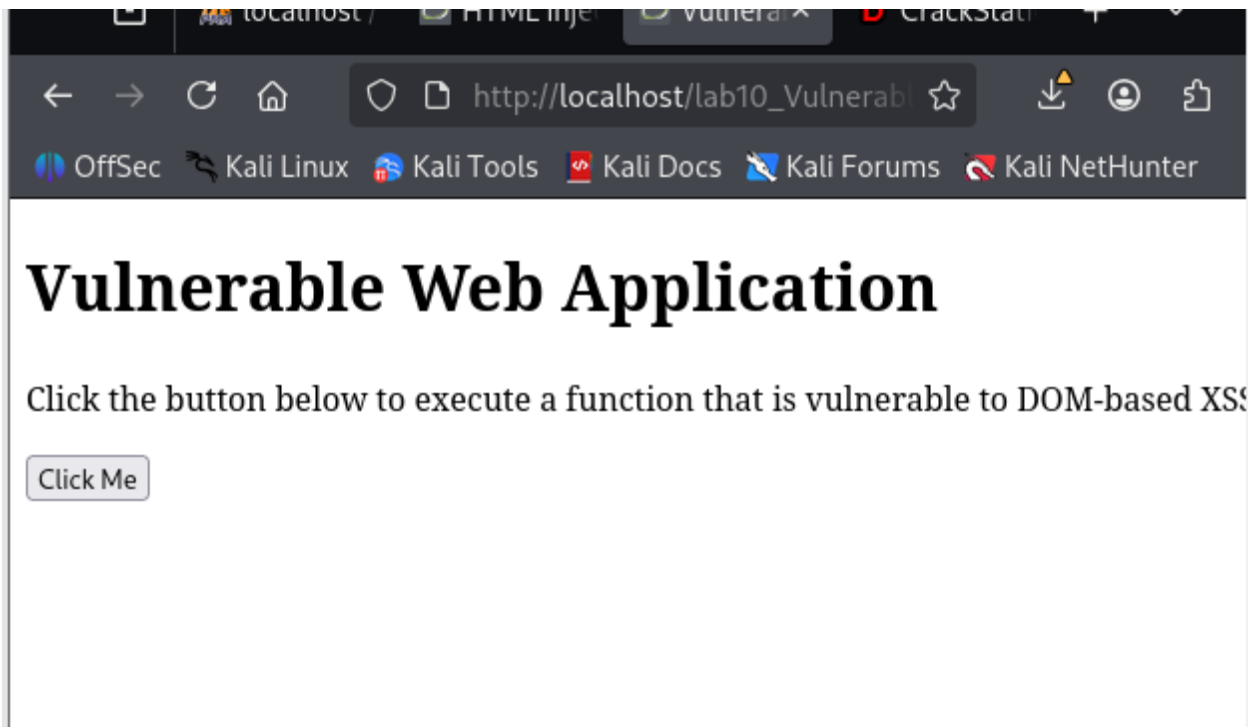
(raph@kali)-[/opt/lampp/htdocs/lab10_VulnerableComponents]
$
```

## Step 2: Exploiting the Vulnerable Components

To test the vulnerability, I opened the web application in my browser and clicked the button on the page. When the input prompt appeared, I entered a malicious script payload:

```
<script>alert('XSS Vulnerability Exploited');</script>
```

After submitting the input, an alert box popped up on the screen. This showed that the script was executed successfully. The reason this happened is because the application was using an old version of jQuery and the `.html()` method, which directly inserts user input into the page without any checks. Since the input was not validated or filtered, the browser treated it as code and executed it, confirming the presence of a DOM-based XSS vulnerability.



# Web Application

Write a function that is vulnerable to DOM-based XSS.

🌐 localhost

Enter some text:

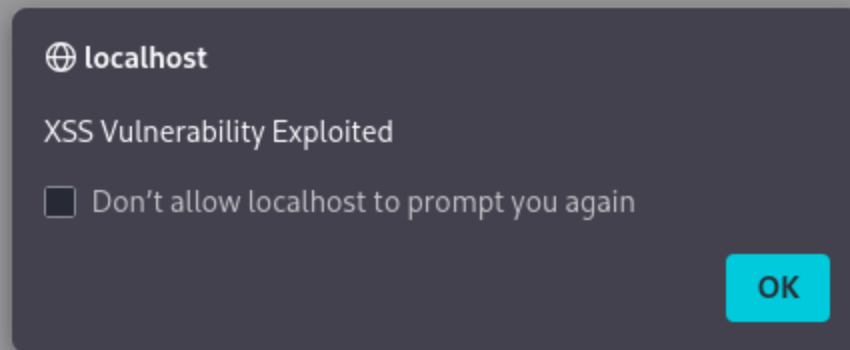
`<script>alert('XSS Vulnerability Exploited');</script>`

Cancel

OK

# Application

unction that is vulnerable to DOM-based XSS.



## Reflection Questions:

- **What are the risks associated with using outdated components like old jQuery versions?**

Using old components is risky because they have known security problems that are not fixed. Attackers already know these weaknesses and can easily take advantage of them. This can lead to issues like running harmful scripts, stealing data, or damaging the website. Old components make the application unsafe.

- **How did the use of an outdated component lead to the exploitation of a vulnerability?**

The vulnerability happened because the old jQuery version did not safely handle user input. The application used the `.html()` method, which allowed user input to be treated as code. Because the component was outdated, the malicious script was executed in the browser, which caused the XSS attack.

## Task 2: Mitigate the Vulnerability

After updating jQuery, I also fixed the insecure part of the code. Instead of using the `.html()` method, which can execute harmful scripts, I used the `.text()` method. This change makes sure that any input entered by the user is treated as plain text and not as code.

By using `.text()`, even if someone enters a script, it will not run in the browser. This helped me prevent the XSS attack and made the application safer to use.

```
Session Actions Edit View Help
GNU nano 8.6 index.php *
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vulnerable Component Example</title>
  <script src="jquery-1.3.2.min.js"></script> <!-- Old jQuery version -->
</head>
<body>
  <h1>Vulnerable Web Application</h1>
  <p>Click the button below to execute a function that is vulnerable to DOM-based XSS.</p>
  <button id="vulnerableBtn">Click Me</button>

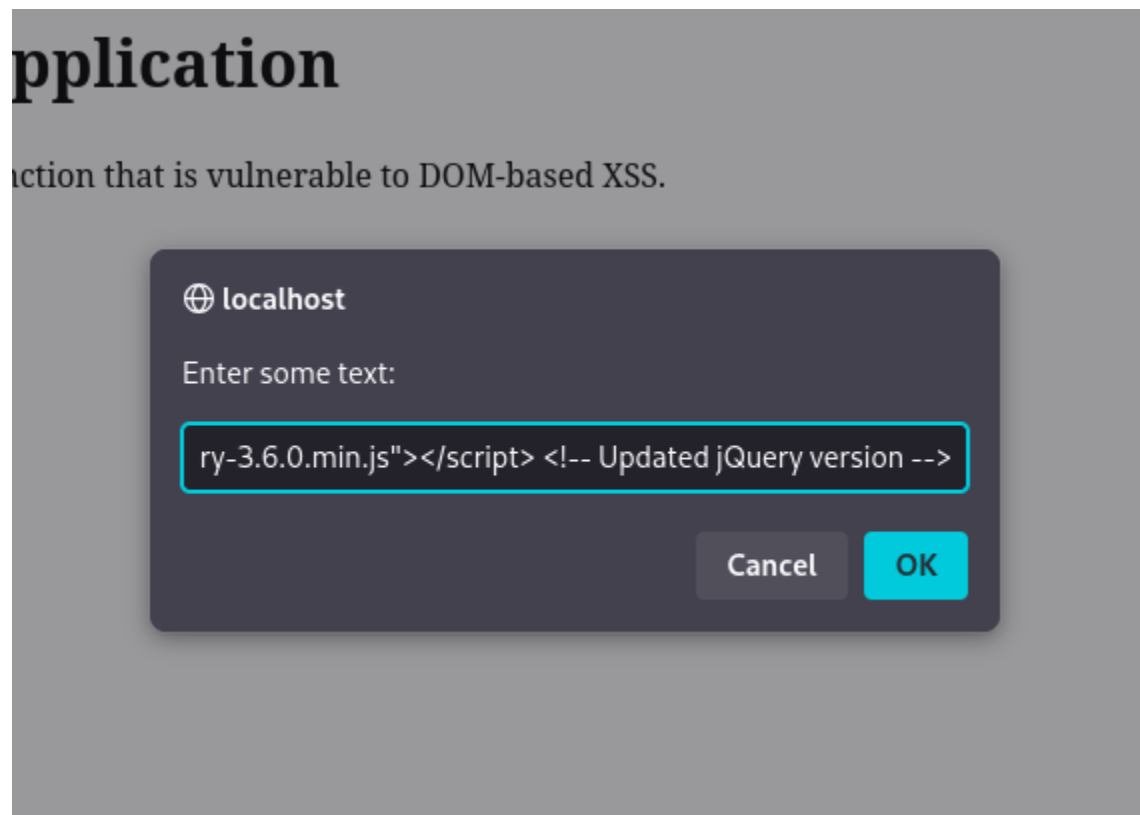
  <script>
    $('#vulnerableBtn').click(function() {
      var userInput = prompt("Enter some text:");
      // Use text() instead of html() to prevent XSS
      $('#vulnerableBtn').text(userInput); // Safely insert text without executing it
    });
  </script>
</html>
</body>
```

## Step 2: Tested the Mitigated Version

After fixing the application, I opened the updated version in my browser and clicked the button again. When the input prompt appeared, I entered the same malicious payload:

```
<script>alert('XSS Vulnerability Exploited');</script>
```

This time, nothing happened. The script did not execute, and the text was displayed as plain text. This showed that the vulnerability had been successfully fixed.





# Vulnerable Web Application

Click the button below to execute a function that is vulnerable to DOM-based XSS.

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script> <!-- Updated jQuery version -->
```

## Reflection Questions:

- How did updating the jQuery version help mitigate the XSS vulnerability?

Updating the jQuery version helped because the newer version includes security fixes that are missing in older versions. It handles user input more safely and reduces the chances of known attacks like XSS. Using the updated version made the application more secure and less vulnerable to exploitation.

- How does using the correct jQuery methods (e.g., `.text()` instead of `.html()`) contribute to secure development?

Using the correct jQuery methods helps because `.text()` treats user input as plain text instead of code. This prevents malicious scripts from running in the browser. By choosing safer methods, I can protect the application from attacks and follow better security practices when developing web applications.

## Index.php

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Vulnerable Component Example</title>

<script src="jquery-1.3.2.min.js"></script> <!-- Old jQuery version -->

</head>

<body>

  <h1>Vulnerable Web Application</h1>

  <p>Click the button below to execute a function that is vulnerable to DOM-based XSS.</p>

  <button id="vulnerableBtn">Click Me</button>


  <script>

    // A vulnerable function that uses jQuery's old .html() method unsafely

    $('#vulnerableBtn').click(function() {

      var userInput = prompt("Enter some text:");

      $('#vulnerableBtn').html(userInput); // Vulnerable to XSS in old jQuery

    });

  </script>

</body>

</html>
```

## Key Takeaway

From this lab, I learned that using outdated components can create serious security risks in a web application. Even a small feature can become dangerous if it uses old libraries or unsafe code. I also learned that updating software and using safe methods to handle user input can prevent common attacks like XSS.

## **Conclusion**

In conclusion, this lab helped me understand how vulnerable components can be exploited and how easily they can be fixed. By updating jQuery and changing the code to handle input safely, I was able to stop the attack from working. This lab showed me the importance of keeping components up to date and following secure coding practices when building web applications.